13-understanding-test-doubles/02/expense_tracker/spec/unit/ledger_double_spec.rb
```
ledger = instance_double('ExpenseTracker::Ledger')
allow(ledger).to receive(:record)
```

With this double in place, RSpec checks that the real Ledger class (if it's loaded) actually responds to the record message with the same signature. If you rename this method to record_expense, or add or remove arguments, your specs will correctly fail until you update your use of the method *and* your test double setup.

Use Verifying Doubles to Catch Problems Earlier



Although your unit specs would have had a false positive here, your acceptance specs would still have caught this regression. That's because they use the real versions of the objects, rather than counting on test doubles.

By using verifying doubles in your unit specs, you get the best of both worlds. You'll catch errors earlier and at less cost, while writing specs that behave correctly when APIs change.

RSpec gives you a few different ways to create verifying doubles, based on what it will use as an interface template for the double:

instance_double('SomeClass')
: Constrains the double's interface using the *instance methods* of SomeClass

class_double('SomeClass')
: Constrains the double's interface using the *class methods* of SomeClass

object_double(some_object)
: Constrains the double's interface using the methods of some_object, rather than a class; handy for dynamic objects that use method_missing

In addition, each of these methods has a _spy variant (such as instance_spy) as a convenience for using a verifying double as a spy.

Stubbed Constants

Test doubles are all about controlling the environment your specs run in: what classes are available, how certain methods behave, and so on. A key piece of that environment is the set of Ruby constants available to your code. With *stubbed constants*, you can replace a constant with a different one for the duration of one example.

For instance, password hashing algorithms are slow by design for security reasons—but you may want to speed them up during testing. Algorithms like